Elastic Stack lab for MyScoutee

A working local Docker lab and an integration decision guide. Code analysis ran against the real MyScoutee checkout; product scenarios use clearly fictional seed data.

Measured baseline, 27 October 2026. The search index contains 700 Java/TypeScript files, 140,898 source lines and 534 heuristic findings. Filebeat-Logstash, Metricbeat and Heartbeat also wrote documents into Elasticsearch.

What does this lab build?

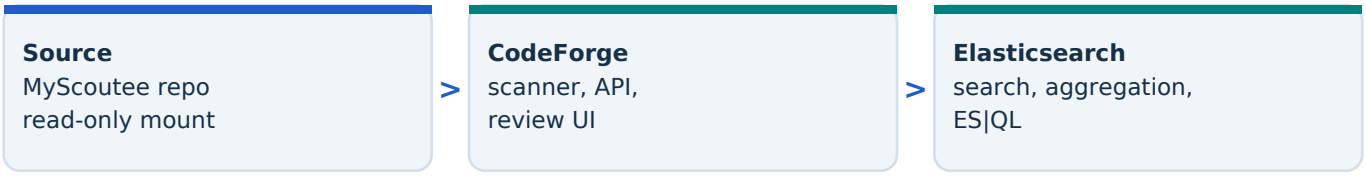


Figure 1. Code analysis is a separate product. The MyScoutee application sends no live user data to it.

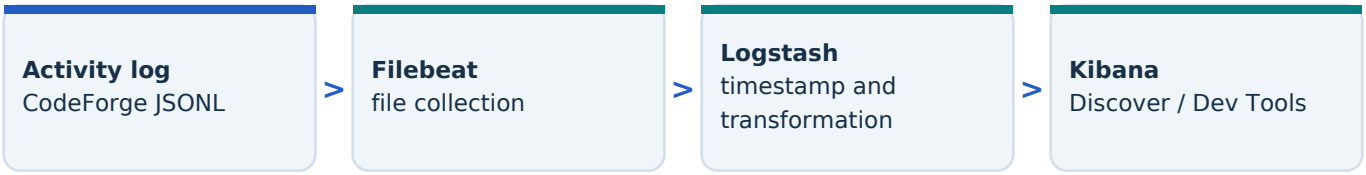


Figure 2. The same lab stores processed events in Elasticsearch and explores them in Kibana.

Goal: exercise Elastic Stack search, processing and observability on real data paths. The lab also preloads fictional offerings, service events and endpoint-shaped signals so Maps, Discover, Security and charts have content on first start.

Scope. The 534 findings come from simple heuristics, not verified defects. This guide does not imply that the MyScoutee backend already connects to Elasticsearch.

Start and verify the lab

Run commands from the myscoutee-elastic repository root. Localhost ports and the read-only source mount are intended for local evaluation.

Run against the real checkout

```
SOURCE_ROOT=../myscoutee-backend PROJECT_NAME=MyScoutee \
bash scripts/core-lab.sh
```

Surface	Address	Use / login
CodeForge	localhost:8088	Search, findings, Stack lab; codeforge login
Kibana Discover	localhost:5601/app/discover	Data views and ES|QL; elastic login
Findings dashboard	localhost:5601/app/dashboards#/view/codeforge-findings-overview	Heatmap, severity pie, module and review charts
Telemetry dashboard	localhost:5601/app/dashboards#/view/codeforge-stack-telemetry	Activity, HTTP checks and metric samples
Elasticsearch	localhost:9200	Authenticated REST API and index checks
Fleet Kibana	localhost:5602/app/fleet/agents	Separate lab; elastic login

Start Fleet with **bash scripts/fleet-lab.sh**. Its prepared System metrics dashboard is at **localhost:5602/app/dashboards#/view/codeforge-fleet-system**. Core passwords are in **.local/core.env**: use ELASTIC_PASSWORD for Kibana/Elasticsearch and CODEFORGE_PASSWORD for CodeForge. This file also holds persistent Kibana encryption keys. **Fleet Kibana username: elastic.** Its password is the ELASTIC_PASSWORD value in the separate **.local/fleet.env**. Both files stay outside Git.

Open prepared views

```
# Log in through the browser, then open Analytics > Dashboards
# Select CodeForge | Findings overview or Stack telemetry
```

Startup prepares eleven Discover data views, four custom dashboards and 24 reusable library charts. The real CodeForge and Stack paths include **forge-files**, **forge-findings**, **forge-activity**, **metricbeat-*** and **heartbeat-***. Rescan or change a finding review state to generate another activity event.

Also open **Analytics > Maps** and select **MyScoutee demo | Service offering map**; **Analytics > Visualize library** contains reusable charts. Three saved Discover sessions and five synthetic rules under **Security > Rules** are prepared. The empty Maps Create page is an editor, not the saved map.

In **Analytics > Dashboards**, open the managed **[Elastic Defend] Endpoint Detection and Response** dashboard. Its endpoint alerts are synthetic; choose **Today** when the 15-minute default range no longer includes them.

> **Bounded scan.** The scanner reads at most 350 files per language and 128 KB per file. Full source text remains in the local Elasticsearch volume; mount confidential repositories only on a trusted machine.

Data paths across the Stack

Statuses combine verified local document counts on 27 October 2026 with the Compose profiles. A running container alone is not proof of a working pipeline.

Component	Lab data / capability	Status
Elasticsearch + Kibana	Code, findings, logs, telemetry, search and visualization	Verified
Demo fixtures	64 fictional offerings; 96 service events; 32 endpoint signals	Seeded on start
Logstash + Filebeat	CodeForge JSONL -> 5044 -> forge-activity	Verified: documents > 0
Metricbeat	Elasticsearch cluster, node and index metrics	Verified: documents > 0
Heartbeat	HTTP probes for CodeForge, ES and Kibana	Verified: documents > 0
Elastic Agent	Standalone CPU and memory metrics	agent profile; verified
Auditbeat + Packetbeat	Source file inventory and CodeForge HTTP traffic	deep profile; verified
Fleet + Winlogbeat	Managed agent; Windows event logs	Fleet verified / Windows host

Optional paths

```
docker compose --env-file .local/core.env --profile agent up -d elastic-agent
SOURCE_ROOT=../myscoutee-backend \
docker compose --env-file .local/core.env --profile deep up -d auditbeat packetbeat

bash scripts/fleet-lab.sh # separate Fleet environment
```

The separate authenticated Fleet lab at **localhost:5602** has a healthy Fleet Server, a managed Agent and System integration metrics. Winlogbeat collects Windows event logs on a Windows host. Inspect optional paths in CodeForge Stack lab, Fleet UI and container logs. [E1-E4]

> **Interpretation.** Observing the stack and indexing MyScoutee business data are distinct steps. The latter needs a data model, authorization rules and a deletion plan.

MyScoutee: service offerings

A concrete candidate for improving the current in-application search. Sources: ServiceOfferingsService, ServiceOfferingsRepository and ServiceOfferingsController. [M1]

Current behavior

ServiceOfferingsService.page loads offerings from MongoDB and then filters in a Java stream by status, category, and substring matches in title and description. It subsequently evaluates profiles, distance, language and visibility, sorts results and paginates. The **discover** path limits results to published offerings and excludes the actor's own offering.

Proposed search index boundary

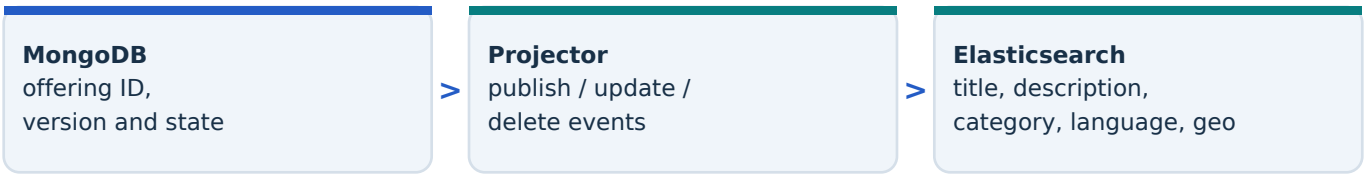


Figure 3. The index is a proposed read model. MongoDB remains authoritative for writes and access decisions.

Experiment	Potential value	Condition
Relevance, typo tolerance, autocomplete	Better ranking for title and description queries	Measure against a real query corpus
Category + distance + language	Filter candidates in one search query	Recheck existing visibility rules
Facets and aggregations	Explore demand and category trends	Do not expose personal data in statistics

Possible replacement. An indexed candidate search could replace the Java substring filter and in-memory sorting of the full offering list. It cannot replace MongoDB storage or the existing profile and access policies.

Try it now: the saved service discovery dashboard, published-offerings Discover session and offering map use the same 64 fictional listings. The coordinates represent example city locations, not MyScoutee users.

MyScoutee: outbox and audit

Operational visibility can be added at existing ownership boundaries. Sources: NotificationOutboxWorkerService and EventTicketAuditService. [M2-M3]

Notification outbox

NotificationOutboxWorkerService watches Mongo change streams and handles due work, leases, retries, delivery windows and Firebase delivery. A trial Elastic stream should contain only structured, minimized transitions: **rule_key, status, attempt, latency_ms, error_class, occurred_at**.

Logstash or Elastic Agent could forward application JSON logs; Kibana can chart delayed and failed deliveries. Alerts could be assessed alongside existing **OperationalTelemetry**. Elastic is not the delivery queue and cannot replace the Mongo outbox, retry logic or Firebase integration.

Ticket check-in replay audit

EventTicketAuditService already records repeated check-in attempts as **check-in-replay** and **already_used** in MongoDB. A filtered secondary index could support searches by event and time window and help investigate spikes in replay attempts.

Source data	Proposed Elastic field	Constraint
Audit identity	audit_id, action, result	Idempotent upsert / deduplication
Event and time	event_id, attempted_at	Retention and access control
Actor and ticket	May be unnecessary	Pseudonymization and authorization

Possible replacement. Elastic/Kibana could replace ad hoc log searches and some administrative summaries after data quality and access checks. The primary Mongo audit record remains the owned evidence.

Try it now: the event reliability dashboard and retry Discover session use 96 synthetic records. A local Security query rule highlights replay failures without external notifications.

Elastic Defend dashboard: four additional demo rules turn 32 synthetic endpoint-shaped signals into local Security alerts. The managed Endpoint Detection and Response dashboard then shows severity, MITRE, OS, host and user panels. Select Today if its 15-minute time range has expired. No host is protected by this fixture.

Integration decision gates

This lab does not change the MyScoutee backend. The following order defines a measured integration trial rather than an unverified migration.

Gate	Action	Evidence
1. Data scope	Choose one search or telemetry case and list required fields	No accidental personal data or secrets
2. Projection	Emit owner-side change signals or use an idempotent projector	Rebuild, deletion and version handling
3. Authorization	Check candidate IDs against existing policy; filter workspace/tenant	Same visible result set as the current API
4. Measurement	Measure P95 latency, relevance, index lag and failures	Compare with the current path
5. Operations	Plan backfill, DLQ, monitoring, retention and backup	Lost or duplicate events handled

Suggested experiment order

First: explore CodeForge and the prepared fictional offering, map and event views. **Second:** export a separate, non-production set of service offerings and measure search quality. **Third:** stream a minimal subset of outbox or ticket-audit event fields. Existing MyScoutee owners define the rules in every step.

> **Critical risk.** An Elasticsearch index may lag or be incomplete. User-visible search results must retain the current access check, and the fallback for a missing index must be defined.

Exercises and evidence

Each exercise connects a component to an observable result. Check returned fields and document counts, not only whether a container is Up.

Exercise	Expected evidence
Search outbox in CodeForge	Source excerpts and related hits in forge-files
Filter by language, module and severity	Aggregated count; finding review state can change
Rescan and inspect forge-activity	scan_completed event with a Logstash-processed timestamp
Request CodeForge and Kibana over HTTP	heartbeat-* monitor event with up/down state
Inspect cluster and index metrics	metricbeat-* document
Open saved offering map and charts	Geo points, city/category facets and reusable library visuals
Open synthetic replay rule	Rule queries demo events; no external action is configured
Open managed Elastic Defend dashboard	32 synthetic endpoint alerts fill the EDR panels; no protected host
Start optional Agent, Auditbeat and Packetbeat	Each path has documents in its own index

Quick diagnosis

```
docker compose --env-file .local/core.env ps -a
docker compose --env-file .local/core.env logs --tail=60 \
logstash filebeat metricbeat heartbeat
```

If **forge-activity** is empty, inspect the CodeForge JSONL log and Filebeat's connection to Logstash port 5044. Optional profiles start separately. **docker compose down -v** deletes local indexes, so use it only when you want to discard them.

On the managed **[Elastic Security] Detection rule monitoring** dashboard, a **(null)** Warning or Failed card means its filter found no matching status changes in the selected time range. Treat it as zero for that status. Succeeded runs are genuine; the lab does not inject failing rules to fill these cards.

> **Completion check.** The core path (source -> search), log path (Filebeat -> Logstash -> search) and telemetry paths (Metricbeat and Heartbeat) must all yield retrievable documents.

Sources and validity

MyScoutee findings reflect the local checkout on 27 October 2026. The integration diagrams are proposals, not completed backend changes.

MyScoutee source locations

[M1] `server/projects/profile/src/main/java/com/raxim/myscoutee/domain/community/service/ServiceOfferingsService.java`; also **repository/ServiceOfferingsRepository.java** and **controller/ServiceOfferingsController.java**.

[M2] `server/projects/profile/src/main/java/com/raxim/myscoutee/domain/notifications/service/NotificationOutboxWorkerService.java`.

[M3] `server/projects/profile/src/main/java/com/raxim/myscoutee/domain/events/service/EventTicketAuditService.java`.

Primary Elastic documentation

[E1] Logstash Docker: https://www.elastic.co/docs/reference/logstash/docker-config

[E2] Filebeat Docker: https://www.elastic.co/docs/reference/beats/filebeat/running-on-docker

[E3] Elastic Agent containers: https://www.elastic.co/docs/reference/fleet/elastic-agent-container

[E4] Fleet Server prerequisites: https://www.elastic.co/docs/reference/fleet/add-fleet-server-on-prem

[E5] Dashboards as code: https://www.elastic.co/docs/explore-analyze/dashboards/create-dashboards-programmatically

[E6] Endpoint Detection and Response dashboard: https://www.elastic.co/docs/solutions/security/dashboards/endpoint-detection-response-dashboard

[E7] Elastic Defend integration: https://www.elastic.co/docs/reference/integrations/endpoint

[E8] Monitor rule executions: https://www.elastic.co/docs/solutions/security/detect-and-alert/monitor-rule-executions

Repository entry points

README.md explains startup and components; **compose.yaml** defines profiles; **config/** contains ingest and telemetry settings. The editable PDF source is **guides/build_elastic_guide.py**.

> **Security boundary.** Core Elasticsearch, Kibana, CodeForge and collectors require credentials. The EDR dashboard uses synthetic alerts from demo-* hosts, not a real Elastic Defend installation. Container traffic remains HTTP without TLS; published ports bind to localhost. Winlogbeat requires Windows. Production needs stronger identity and transport security.